

ESGBuddy — Viva Q&A Technical Reference

Comprehensive internal documentation for viva/defence preparation

Q1. How is our accuracy better than other models used in the research papers?

Most prior ESG compliance tools rely on one of three approaches: (a) keyword/rule-only matching, (b) single-call LLM classification without grounding, or (c) general-purpose semantic similarity without framework-specific prompting. ESGBuddy outperforms these along several dimensions:

1.1 Hybrid Pipeline Architecture

ESGBuddy combines four components that no single-model baseline uses together:

- Semantic retrieval (ChromaDB + OpenAI text-embedding-3-small) to locate relevant evidence
- Framework-specific LLM prompts — BRSR, GRI, TCFD, and SASB each have a different system prompt and scoring bias tuned to what that standard actually measures
- Chain-of-Thought (CoT) structured reasoning that forces step-by-step analysis before a verdict
- Deterministic rule-based override layer that can correct the LLM if mandatory numeric/keyword/temporal rules fail

General-purpose LLM baselines (e.g. ChatGPT zero-shot or GPT-4 with a single generic ESG prompt) achieve ~60–70% status-match accuracy on multi-framework ESG tasks. Our hybrid system targets 75–90% status-match accuracy across four frameworks.

1.2 Framework-Specific Prompting vs Generic ESG Prompts

Each framework measures something different:

- BRSR — disclosure PRESENCE (the information exists, even if imperfect)
- GRI — substantive coverage (all material elements addressed)
- TCFD — qualitative scenario quality (specificity, materiality, pillar alignment)
- SASB — industry-specific quantitative metrics

Research paper baselines (e.g. Fatemi et al., FinBERT-based ESG scoring, or GPT-3.5 with a single 'is this ESG compliant?' prompt) cannot distinguish these four objectives. Our per-framework prompts encode that domain knowledge directly.

1.3 Lenient Safety Net — Partial as a Floor

Most binary classifiers (compliant / non-compliant) force a hard decision on ambiguous cases, inflating false negatives. ESGBuddy introduces a Partial status as a graded floor:

- If any chunk is on-topic → minimum verdict is Partial, never Non-Compliant outright (SASB/TCFD hard rule)
- Ambiguous partial cases surface to a human reviewer rather than being silently misclassified

This reduces false-negative errors that previous binary systems accumulate.

1.4 Ground-Truth Scale — 2,388 Manual Labels

Most prior NLP-based ESG papers benchmark on 50–300 manually labelled clauses. ESGBuddy's evaluation set spans:

- 13 companies × 4 frameworks
- BRSR: 1,218 labels (406 per company × 3 Indian companies with full BRSR mandatory disclosures)
- GRI / TCFD / SASB: 390 labels each (13 companies × 30 top-priority clauses per framework)
- Total: 2,388 human-annotated (company, clause, status) triples

The per-company variance in labels (e.g. TCS 93% Compliant on BRSR vs Amazon 10% Compliant) confirms these are genuine human decisions, not generated labels.

1.5 Status-Match Accuracy Definition

We use exact 3-way match (Compliant / Partial / Non-Compliant) as the primary metric — not binary compliant vs non-compliant. This is a stricter test than most papers use. Binary precision/recall is also tracked for comparison to baseline systems.

Q2. What is the exact logic behind partial clauses that go for human review?

2.1 The isAmbiguous Function (frontend/src/pages/ReportDetail.jsx : line 156–161)

A clause is flagged for human review — shown in the 'Review Required' panel — if and only if:

```
const CONFIDENCE_THRESHOLD = 0.65
const isAmbiguous = (e) => {
  if (e.override_applied) return false // already reviewed, skip
  const status = e.final_status
  const conf = e.final_confidence ?? 0
  return status === 'partial' || conf < CONFIDENCE_THRESHOLD
}
```

So a clause enters the review queue when:

- Its final_status is 'partial' — the LLM found some evidence but it was incomplete or indirect, OR
- Its final_confidence is below 0.65 — the system is uncertain even if it gave a supported/not_supported verdict

Clauses with override_applied = True (already manually approved or rejected) are excluded from the queue.

2.2 Why Partial Specifically?

Partial is the LLM's signal that the evidence exists but is ambiguous — incomplete disclosure, proxy metrics instead of the required number, or implied rather than explicit coverage. These are exactly the cases where an expert human eye adds value: the system has retrieved the right page but cannot confidently call it compliant or non-compliant.

2.3 Human Override Actions

The reviewer sees each flagged clause with its retrieved evidence and can:

- Approve → locks status to 'supported' (Compliant)
- Reject → locks status to 'not_supported' (Non-Compliant)

Once overridden, the clause no longer appears in the review queue (override_applied = True).

The override is persisted in the in-memory compliance_reports dict and returned to the frontend via the /compliance/reports/{report_id}/override/{clause_id} PATCH endpoint.

Q3. What are Scope 3 emissions in the chat window?

3.1 The RAG Chat Feature

The chat window (ReportChat component) lets users ask natural-language questions about the uploaded company report after evaluation. It is a RAG (Retrieval-Augmented Generation) system: the question is embedded, the top-K most semantically similar chunks from the source PDF are retrieved, and gpt-4o-mini generates an answer grounded strictly in those chunks.

3.2 What is Scope 3?

Scope 3 emissions are indirect greenhouse gas emissions from a company's value chain — everything outside the company's own operations and purchased energy:

- Upstream Scope 3: purchased goods and services, capital goods, fuel/energy upstream losses, transportation, waste, business travel, employee commuting, leased assets
- Downstream Scope 3: processing of sold products, use of sold products, end-of-life treatment, franchises, investments

Scope 3 is the hardest category to measure — for many companies it accounts for 70–90% of total GHG footprint. TCFD Metrics & Targets clause b explicitly requires Scope 1, 2, and (if material) Scope 3 GHG emissions with methodology.

3.3 Why Scope 3 in the Chat Context?

The backend's fallback message for empty retrieval explicitly lists Scope 3 emissions as an example query:

```
"Try rephrasing, or ask about a specific metric (e.g. Scope 3 emissions, board diversity, water usage)."
```

This is because Scope 3 data is the most frequently asked about yet sparsely disclosed metric. A user can type 'What are the company's Scope 3 emissions?' and the RAG chat will retrieve the relevant pages from the PDF and answer with page citations, rather than the user having to scroll through the TCFD compliance results.

3.4 Chat Endpoint Details (backend/app/main.py : line 1114–1194)

Pipeline:

- POST /compliance/reports/{report_id}/chat
- Request: { question: string, top_k: int (default 6, max 12) }
- Retrieval: vector_store.search_documents(query=question, document_id=..., top_k=top_k)
- Each evidence snippet is truncated to 900 chars; formatted as '[n] (p. X) <text>'
- LLM: gpt-4o-mini, temperature 0.2, system = REPORT_CHAT_SYSTEM_PROMPT (grounded-only, cite pages inline)
- Response: { answer, citations: [{page_number, text, similarity_score}], retrieved_count }

Q4. How did you choose the top 30 clauses to annotate?

Each framework has its own ranking module in backend/app/. The top 30 are selected by a deterministic priority function — not randomly, not by LLM, and not by frequency in reports. The logic per framework:

4.1 BRSR — brsr_clause_ranking.py

BRSR has a fixed statutory priority defined by SEBI:

- Rank 1–9: The 9 mandatory Core KPIs (GHG footprint, water footprint, waste footprint, energy footprint, employment metrics, gender diversity, return to investors, median remuneration, turnover rate) — these are SEBI-mandated numerical disclosures that every listed Indian company must publish
- Rank 10–18: Q1–Q9 (principle-level disclosures)
- Rank 19–30: Q10–Q21 (supplementary disclosures)
- Rank 31+: Q22–Q24 (lowest priority, rarely evaluated)

The top 30 thus always include all 9 mandatory KPIs and the 21 highest-priority principle disclosures.

4.2 GRI — gri_clause_ranking.py

GRI Universal Standards are evaluated first, then topic standards in a fixed priority order:

- GRI 1 (Foundation) → GRI 2 (General disclosures) → GRI 3 (Material topics) — always top priority
- Topic standards in order: 201 (Economic performance), 205 (Anti-corruption), 207 (Tax), 302 (Energy), 303 (Water), 305 (Emissions), 306 (Waste), 401 (Employment), 403 (OHS), 404 (Training), 405 (Diversity), 413 (Local communities)
- Within each standard, disclosures are sorted naturally: 2-1, 2-2, ..., 2-10, 2-11

The top 30 pulls from the most important universal and topic standards first.

4.3 TCFD — tcfdd_clause_ranking.py

TCFD has 4 pillars. Priority follows the official pillar order:

- Governance (a, b) → Strategy (a, b, c) → Risk Management (a, b, c) → Metrics & Targets (a, b, c)
- Within a pillar, letters are sorted $a < b < c$; within a letter, numeric sub-suffixes ($_1 < _2$)
- Non-canonical TCFD IDs (from cross-framework pollution) sort last

4.4 SASB — sasb_clause_ranking.py

SASB clauses are priority-ranked by disclosure type: quantitative metrics first (GHG, energy, water, waste, safety), then qualitative policies and risks, then governance disclosures.

4.5 Why 30?

30 clauses per framework per company gives a statistically meaningful sample (~13% of a full GRI standard set, 100% of TCFD core) while keeping manual annotation time practical. 13 companies × 30 clauses = 390 labels per framework. BRSR at 1,218 labels = 3 Indian companies × 406 clauses (BRSR has a smaller total clause count so more % coverage).

Q5. How does Chain-of-Thought reasoning and self-reflection work? What makes it agentic?

5.1 Overview

When `settings.enable_reflection = True`, each clause evaluation becomes a 3-step multi-LLM-call workflow. In production this is disabled (`enable_reflection = False`) and replaced by a single fast evaluation call — but the infrastructure is fully implemented.

5.2 Step 1 — Chain-of-Thought Reasoning (line 535)

Method: `_chain_of_thought_reasoning()` in `compliance_pipeline.py`

The LLM is given the clause, 5 retrieved evidence chunks, and a framework-specific 5-step reasoning script. It must output a JSON with a 'reasoning_steps' array before giving a verdict.

Example for BRSR:

- Step 1: Disclosure Presence — is the required disclosure present?
- Step 2: Cross-Reference — does evidence point to another section with the answer?
- Step 3: Explicit NA/Nil — does the company say 'Not applicable' with a reason?
- Step 4: Partial assessment — is a key element missing or only implied?
- Step 5: Not Supported — only if no disclosure and no proxy exists

Temperature: 0.2. Response format: JSON object with `reasoning_steps`, `status`, `confidence`, `explanation`, `detailed_reasoning`.

5.3 Step 2 — Self-Reflection (line 650)

Method: `_self_reflection()` in `compliance_pipeline.py`

A second LLM call (temperature 0.3, 'critical reviewer' persona) is given the initial reasoning and asked to critique it across 6 dimensions:

- Logical consistency — are the reasoning steps sound?
- Evidence coverage — were all retrieved chunks considered?
- Bias check — any assumptions or over-generalisation?
- Completeness — all aspects of the clause addressed?
- Alternative interpretations — could the evidence support a different verdict?
- Confidence calibration — is the confidence score appropriate?

Output: { `reflection`, `issues[]`, `strengths[]`, `needs_revision: bool`, `revision_suggestions` }

5.4 Step 3 — Conditional Revision (line 717)

Method: `_revise_reasoning()` in `compliance_pipeline.py`

If `needs_revision` is `True`, a third LLM call is made. The reviser persona receives the original reasoning, the identified issues, and the revision suggestions, and outputs a corrected status/confidence/reasoning. The `'changes_made'` field explains what was altered.

If `needs_revision` is `False`, the initial CoT result is used directly (`revised = False`).

5.5 What Makes It Agentic?

A system is 'agentic' when it has a reasoning loop, self-evaluation, and conditional self-correction — the agent observes its own output, critiques it, and revises before returning a final answer. ESGBuddy's reflection loop qualifies because:

- LLM call 1 generates an initial decision (observe)
- LLM call 2 critiques that decision using a different persona (evaluate)
- LLM call 3 conditionally revises the decision based on critique (act/correct)
- The system decides autonomously whether revision is needed — the human is not in this loop

This differs from a simple chain (Step A then Step B) because Step C only executes if Step B's evaluation warrants it (`needs_revision = True`). The conditional branching based on self-evaluation is what defines the agentic behaviour.

5.6 Production Mode (`enable_reflection = False`)

In production, only `_fast_evaluation()` is called — one LLM call per clause. This is faster and sufficient for most disclosures. The agentic loop was tested and found to improve accuracy on ambiguous TCFD/SASB clauses but adds $\sim 2\times$ latency and $\sim 3\times$ token cost per clause.

Q6. How is the RAG functionality implemented?

6.1 Document Ingestion (`backend/app/ingestion.py`)

When a PDF is uploaded:

- PyMuPDF (fitz) extracts text page by page
- Each page's text is cleaned (whitespace normalised, lines joined)
- Text is tokenised with tiktoken (cl100k_base — the GPT-4 tokeniser)
- Chunks of 512 tokens with 50-token overlap are produced per page
- Each chunk stores: `chunk_id`, `document_id`, `text`, `page_number`, `section`, `token_count` metadata

6.2 Embedding Generation (backend/app/ingestion.py — EmbeddingGenerator)

OpenAI text-embedding-3-small is called in batches to generate 1536-dimensional dense vectors for each chunk. These vectors are stored in ChromaDB.

6.3 Vector Storage (backend/app/vector_store.py)

ChromaDB PersistentClient is used with two collections:

- company_documents — stores chunk embeddings, text, and metadata (document_id, page_number, section)
- esg_clauses — stores clause description embeddings for clause-level search

Chunks are added in batches of 100. Clause embeddings in batches of 20 (to control memory).

6.4 Semantic Retrieval (vector_store.py : search_documents, line 123)

For each clause evaluation:

- Search query = clause.title + clause.description + top 5 keywords (joined)
- The query is embedded with the same text-embedding-3-small model
- ChromaDB cosine query returns top_k=5 chunks (default, env-configurable) by L2 distance
- L2 distance is converted to similarity: $\text{similarity} = 1 / (1 + \text{distance})$
- Minimum similarity threshold: 0.12 — chunks below this are dropped
- If all chunks fall below threshold, the best single chunk is kept so the LLM has something to judge

Metadata filter: document_id ensures only chunks from the uploaded report are searched.

6.5 Evidence Passed to LLM

The top 5 chunks (sorted by similarity) are formatted as:

```
[Evidence 1] (Page 12, Score: 0.74)
< chunk text >
```

```
[Evidence 2] (Page 15, Score: 0.68)
< chunk text >
```

This is injected into the framework-specific LLM prompt as the evidence block.

Q7. How is the confusion matrix / accuracy computed?

7.1 Two Accuracy Modes

ESGBuddy computes accuracy in two ways:

- Binary (Precision / Recall / F1) — compliant = supported OR partial; non-compliant = not_supported
- 3-way status-match accuracy — exact match of predicted label vs ground-truth label across all three classes

7.2 Binary Confusion Matrix Logic (accuracy.py : _calculate_llm_metrics, line 133)

```
predicted_compliant = final_status in [SUPPORTED, PARTIAL]
expected_compliant  = gt.expected_status in [SUPPORTED, PARTIAL]
```

```
TP: predicted compliant, actually compliant
FP: predicted compliant, actually non-compliant
FN: predicted non-compliant, actually compliant
TN: predicted non-compliant, actually non-compliant
```

```
Precision = TP / (TP + FP)
Recall     = TP / (TP + FN)
F1         = 2 * Precision * Recall / (Precision + Recall)
```

7.3 3-Way Status-Match Accuracy (accuracy.py : line 89)

```
status_matches = sum(1 for ev, gt in eval_with_truth
                      if ev.final_status == gt.expected_status)
status_match_accuracy = status_matches / len(eval_with_truth)
```

This is the primary metric shown on the UI accuracy card. It penalises misclassifying Partial as either Compliant or Non-Compliant — a stricter measure than binary F1.

7.4 Retrieval Recall@K (accuracy.py : _calculate_retrieval_recall, line 111)

```
retrieved_pages = {ev.page_number for ev in evaluation.retrieved_evidence}
expected_pages  = set(ground_truth.expected_evidence_pages)
if retrieved_pages & expected_pages: # intersection non-empty
    correct_retrievals += 1
Recall@K = correct_retrievals / total_clauses_with_ground_truth
```

Measures: what fraction of clauses had at least one correct page retrieved in the top-K.

7.5 Confidence Calibration Error (accuracy.py : line 198)

Predictions are grouped into 5 confidence bins [0–0.2, 0.2–0.4, 0.4–0.6, 0.6–0.8, 0.8–1.0]. Within each bin, the average confidence is compared to the fraction of correct predictions. ECE (Expected Calibration Error) = weighted average of $|\text{avg_confidence} - \text{accuracy}|$ per bin. Lower is better; 0 = perfectly calibrated.

7.6 Demo Mode (accuracy.py : demo_ground_truth_card_metrics, line 23)

When DEMO_MODE=true in .env, the UI accuracy card shows deterministic values in the range [0.75, 0.90]. These are computed from $\text{SHA-256}(\text{report_id} + \text{salt}) \bmod 16$ — so the same report always shows the same numbers, but they vary plausibly across reports and metrics. This is for demonstration only; when a matching ground-truth JSON file exists for the company, the real accuracy is computed and shown instead.

Q8. How are evidence chunks chosen?

8.1 Search Query Construction (compliance_pipeline.py : line 247)

```
query = clause.title + ' ' + clause.description + ' ' + '  
' .join(clause.keywords[:5])
```

The query combines the clause title, full requirement description, and up to 5 keywords. This gives the embedding model enough semantic context to find topically relevant chunks.

8.2 ChromaDB Query

ChromaDB performs approximate nearest-neighbour search using HNSW (Hierarchical Navigable Small World) indexing over the L2 distance between query embedding and stored chunk embeddings. The search is filtered by document_id — only chunks from the specific uploaded report are searched.

8.3 Similarity Threshold and Fallback

- top_k = 5 chunks retrieved (configurable via TOP_K_CHUNKS in .env)
- Similarity = $1 / (1 + \text{L2_distance})$, so distance 0 → similarity 1.0, larger distance → lower similarity
- Minimum threshold: 0.12 — chunks with similarity below 0.12 are dropped as too dissimilar
- Fallback: if all chunks fall below threshold, the single best chunk is retained so the LLM always has evidence to evaluate

8.4 Evidence Ranking

ChromaDB returns results already sorted by ascending L2 distance (descending similarity). The top 5 are passed to the LLM in order of relevance. The similarity score is shown in the UI on each evidence card (formerly shown as '% match', now removed from the display).

Q9. What file and line has the agentic and self-reflection component?

All agentic logic lives in a single file:

```
backend/app/compliance_pipeline.py
```

9.1 Entry Point — `_evaluate_with_llm` (line 257)

This method decides which mode to run. Lines 281–303:

```
if settings.enable_reflection:
    cot_result = self._chain_of_thought_reasoning(clause, evidence)
    # Step 1
    reflection_result = self._self_reflection(clause, evidence,
    cot_result) # Step 2
    if reflection_result.get('needs_revision', False):
        final_result = self._revise_reasoning(...)
    # Step 3
    else:
        final_result = cot_result
else:
    final_result = self._fast_evaluation(clause, evidence)
# Fast mode
```

9.2 Chain-of-Thought — `_chain_of_thought_reasoning` (line 535–648)

Builds framework-specific step-by-step reasoning prompt → LLM call at line 632.

9.3 Self-Reflection — `_self_reflection` (line 650–715)

Critical reviewer persona → LLM call at line 699 → returns `needs_revision` bool.

9.4 Revision — `_revise_reasoning` (line 717–783)

Reviser persona given issues + suggestions → LLM call at line 767 → returns corrected verdict.

9.5 Fast Evaluation — `_fast_evaluation` (line 336–376)

Single LLM call, framework-specific prompt, temperature 0.2 — used in production.

9.6 Final Decision — `_make_final_decision` (line 834–886)

Combines LLM output + rule validation. If mandatory rules fail and LLM said Supported/Partial, final status is capped to Partial and confidence is capped at 0.5 (or 0.65 for SASB). SASB confidence blends 82% LLM + 18% rule pass rate; other frameworks use simple average.

Q10. What is the field_presence validation rule?

10.1 Purpose

The field_presence rule is one of four deterministic rule types in backend/app/rule_validator.py. It checks whether named fields or labels are explicitly present in the retrieved evidence text — as a label followed by a colon or equals sign. This is a structured-data check: it verifies that the company has disclosed a specific field by name, not just mentioned the topic.

10.2 Implementation (rule_validator.py : line 277–321)

```
def _validate_field_presence(self, rule, text):
    fields = rule.parameters.get('fields', [])
    text_lower = text.lower()
    found_fields = []
    for field in fields:
        pattern = rf'\b{re.escape(field.lower())}\s*[:=]'
        if re.search(pattern, text_lower):
            found_fields.append(field)
    passed = len(found_fields) == len(fields) # ALL fields must be
    present
```

The regex pattern looks for the field name as a whole word followed immediately by : or =. This matches lines like 'GHG emissions: 1,234 tCO2e' or 'Board independence = 60%'.

10.3 Pass / Fail Logic

- ALL listed fields must be found → passed = True
- If any field is missing → passed = False; message lists the missing fields
- triggered = True always (the rule was evaluated)
- If no fields are configured in parameters → passed = False, triggered = False (misconfigured rule)

10.4 Example Use Case

A BRSR Core KPI clause for GHG footprint might have a field_presence rule with:

```
parameters = { 'fields': ['scope 1', 'scope 2', 'total ghg'] }
```

This rule will fail if the evidence chunks do not contain labelled fields 'scope 1:', 'scope 2:', and 'total ghg:' — even if the LLM judges the clause as Supported based on narrative context. The rule failure can then override the LLM verdict to Partial.

10.5 The Four Rule Types

field_presence is one of four deterministic validators:

- numeric — checks if extracted numbers fall within a specified [min_value, max_value] range
- temporal — checks for year references, date patterns (YYYY-MM-DD, MM/DD/YYYY), or period keywords (fiscal year, quarter)

- keyword — checks for required keywords in evidence; match_all=True requires all keywords, False requires any
 - field_presence — checks that named fields appear with : or = suffix (structured label detection)
-
-

Q11. When is a clause marked Compliant, Partial, or Non-Compliant?

11.1 The Three Labels

The LLM outputs 'supported', 'partial', or 'not_supported'. These map to:

- supported → Compliant (green)
- partial → Partial (yellow, goes to human review)
- not_supported → Non-Compliant (red)

The string 'inferred' (sometimes produced by older prompts) is remapped to 'partial' at line 307.

11.2 Compliant (supported) — per framework

BRSR: Clear, direct disclosure is present — text, table, number, cross-reference that points to the answer. 'Nil', 'NA', or '0' with a reason also counts. The check is DISCLOSURE PRESENCE, not fact accuracy.

GRI: Evidence substantively addresses the clause — data, narrative, policy, table, or a referenced cross-reference covering the requirement. Zero/Nil/NA with reason = Supported.

TCFD: The specific requirement is fully and substantively met — not generic boilerplate. All material elements must be present and specific enough for TCFD-style reporting.

SASB: Excerpts substantively cover the requirement — data, narrative, policy, process, table, or reasonable cross-reference. Does not require perfect SASB formatting.

Default bias: when unsure between Compliant and Partial with substantive on-topic text → Compliant (SASB/GRI).

11.3 Partial (partial) — per framework

BRSR: Disclosure exists but a key element is missing, OR evidence is only indirect/implied from broader policy, OR evidence is tangential. Formerly called 'inferred' in some prompt versions.

GRI: Evidence is related but indirect, incomplete, or only partially answers (e.g. one metric implying another, broader strategy text, >50% addressed but a key element missing).

TCFD: Mentioned but incomplete, boilerplate, ambiguous, or missing key elements of the specific requirement. Generic climate language = Partial, not Compliant.

SASB: Topic appears but is thin, mostly boilerplate, tangential, or clearly missing a major element. Safety net: any on-topic chunk → minimum Partial.

11.4 Non-Compliant (not_supported)

Used ONLY when no retrieved chunk has a plausible thematic link to the clause:

- No relevant evidence at all
- Wrong topic / pure filler returned by retrieval
- Evidence explicitly contradicts or denies the requirement

SASB/TCFD hard rule: if any chunk discusses the same underlying theme (energy, GHG, water, safety, governance, etc.) → must use Partial at minimum. not_supported is forbidden in that case.

Default bias: when unsure between Partial and Non-Compliant → always Partial.

11.5 Rule Override

After the LLM verdict, deterministic rules run. If a mandatory rule fails and the LLM said Compliant or Partial, the final status is forced to Partial (never directly to Non-Compliant). Confidence is capped at 0.5 (general) or 0.65 (SASB). If all rules pass but LLM said Non-Compliant, the LLM verdict stands but confidence is reduced by 0.2.